

Design a Vehicle Rental System

Object-Oriented Programming with C++ | 2 Hours | Intermediate Level

1. The Scenario

A vehicle rental company manages a fleet of vehicles — cars, motorbikes, and trucks. Customers can rent any available vehicle for a specified number of days. Your task is to design and implement a C++ program that models this system.

The scenario is the same for every student. How you design the solution is entirely up to you.

2. What Your Program Must Do

Your solution must satisfy all six requirements below. These are behavioural requirements — they tell you what to achieve, not how to structure your code.

R1 — Manage a fleet of vehicles

- The system must hold multiple vehicles simultaneously.
- Each vehicle has: a make/model name, a daily rental rate (cost per day), and an availability status (available or currently rented).
- The fleet must support at least three vehicle types: Car, Motorbike, and Truck.
- Each vehicle type may have type-specific attributes (e.g. a Car has a number of seats; a Truck has a payload capacity in tonnes).

R2 — Register customers

- A customer has a name and a unique customer ID.
- A customer may hold at most one active rental at a time.

R3 — Process a rental

- A rental assigns one available vehicle to one customer for N days.
- Once rented, the vehicle must be marked unavailable until returned.
- The system must prevent renting an already-rented vehicle.

R4 — Calculate rental cost

- Total cost = daily rate x number of days.
- Trucks incur a 20% surcharge on top of the daily rate (overweight handling fee).
- Motorbikes receive a 10% discount for rentals longer than 7 days.

R5 — Return a vehicle

- Returning a vehicle marks it available again.
- The rental record must reflect the return (the record is not deleted — it should show it is closed).

R6 — Print a summary

- At any point, the system must be able to print: all current active rentals (customer, vehicle, days, cost) and a count of available vs rented vehicles.

3. What You Must Design Yourself

The requirements above say what the system does. They say nothing about how to structure the code. The following design decisions are yours to make. There is no single correct answer for any of them — but you must justify every choice you make.

01	How do you model the three vehicle types? Is Car a subclass of Vehicle, or is the type just a field? When would the other approach be better?
02	Where does the pricing logic live? Is calculateCost() a virtual method on Vehicle, a free function, or a method on Rental? Each placement has consequences — pick one and own it.
03	Who owns the rental record? Does Customer hold it? Does a central RentalSystem manage all rentals? Does Vehicle know it is currently rented? Design the ownership clearly.
04	How many classes do you need? There is no required number. Think about whether a Fleet class adds value, whether Rental deserves its own class, and whether a separate class improves or obscures your design.
05	Where does runtime polymorphism actually appear? It must appear somewhere meaningful — not just as a technicality. Identify the exact line in your main() where a base pointer calls a virtual method and explain why that is the right place for it.

4. What to Submit

4a — C++ source code

- All .cpp and .h files needed to compile and run your solution.
- Code must compile without errors under C++17.
- main() must demonstrate a complete scenario: add at least 3 vehicles, register 2 customers, process 2 rentals, return 1 vehicle, and print the summary.

4b — Design note (written before you begin coding)

Before writing any code, answer the following four questions on paper or in a separate .txt file. This is not optional — it will be read alongside your code and is part of your grade.

5. Marking Rubric

This lab is marked on design reasoning, not feature count. A small, well-designed system whose author can fully justify every decision will outscore a large, broken system. Each criterion is scored 1–4 and then weighted.

4 — Excellent		3 — Good		2 — Developing		1 — Inadequate	
Criterion	Excellent (4)	Good (3)	Developing (2)	Inadequate (1)			
OOP Design (x4 weight)	All OOP concepts applied correctly and purposefully. Inheritance used only where is-a holds. Polymorphism drives real behaviour. Design is clean and coherent.	Most OOP concepts applied correctly. Minor misuse (e.g. shallow inheritance). Polymorphism present but not fully leveraged.	OOP concepts present but misapplied. Inheritance used for code reuse rather than type relationships. Little real polymorphism.	OOP concepts missing or fundamentally wrong. Classes are structs with functions. No meaningful hierarchy or polymorphism.			
Design Justification (x3 weight)	Every key decision explained clearly. Student can articulate why they chose is-a vs has-a, where virtual methods sit, and what was deliberately left out.	Most decisions justified. Some choices unexplained or weakly argued. Student understands their own design but misses nuances.	Superficial justification. Describes what the code does rather than why decisions were made. Cannot explain tradeoffs.	No meaningful justification. Student cannot explain design choices or contradicts their own code.			
Encapsulation & Responsibility (x2 weight)	Each class has a single, clear responsibility. No public data. Access control is deliberate. No God classes.	Mostly well-encapsulated. One or two classes doing too much or minor access control slips.	Some encapsulation but inconsistent. Data partially public. Classes unclear in responsibility.	No encapsulation. All data public. Classes are monolithic or arbitrary.			
Working Demo (x2 weight)	Code compiles cleanly. main() is a meaningful scenario showing all features. Output is readable and demonstrates the system.	Code compiles. main() exercises most features. Minor output issues or missing edge cases.	Code compiles with warnings or minor fixes needed. main() is incomplete or does not demonstrate all requirements.	Code does not compile or produces no meaningful output.			
Code Quality (x1 weight)	Clean, readable code. Consistent naming. No memory leaks. Proper use of const, references. Header/source split.	Mostly readable. Minor inconsistencies in naming or style. Small memory management issues.	Code is hard to follow. Naming is inconsistent. Memory management is absent or wrong.	Code is unreadable, uncommented, or shows fundamental misunderstanding of C++ syntax.			

Score Sheet

Use this table to record scores. Multiply each score by its weight, then sum for the weighted total.

Criterion	Weight	Score (1-4)
OOP Design	x 4	___ / 4
Design Justification	x 3	___ / 4
Encapsulation & Responsibility	x 2	___ / 4
Working Demo	x 2	___ / 4
Code Quality	x 1	___ / 4
Weighted Total (max 48)		___ / 48

Grade conversion: 48 = 100% | 36 = 75% | 24 = 50% | 12 = 25%

Evaluator comments:

6. Notes for the Evaluator

This is an open-ended problem. Two students may submit completely different class structures and both may be fully correct. When marking, ask:

- Does the design reflect deliberate decisions, or does it look like the student stumbled into a working solution?
- Can the student explain the tradeoff between inheritance and composition for vehicle types?
- Is polymorphism used because it solves a real problem in this design, or is it bolted on to satisfy the requirement?
- Does the design note match the code? Contradiction between the two is itself a design error.
- Did the student scope reasonably? A student who built three classes that work and explains them perfectly is better than one who built eight classes half of which are broken.

Important: do not penalise a student for not using a particular OOP feature if they can demonstrate a valid reason why it was not needed in their design. The goal is to assess understanding, not checklist completion.